Last Transmission: Binary Exploit

Creator: Nishant Patel Points: 500

Description

Competitors find a format-string bug in a stripped binary's authentication step, leak a stack address, then compute the real (PIE-randomized) address of a hidden function to hijack control flow and print the flag. Two decoy functions that also open and print files must be ruled out by checking which file each one actually reads

Github Path: Capture_The_Flame_2027/Binary_Exploit/Last_Transmission

Prompt

Deep-space relay station ORION-7 went silent eighteen years ago. Last night its carrier wave came back faint, corrupted, looping a single fragment of what appears to be a final transmission. The relay's old intake process is still running and still listening. It authenticates its operators first, then takes the signal. It was never meant to handle either this closely. Recover the last transmission.

nc <HOST> 31337

Flag format: flame{...}

Hints

1. checksec the binary. No canary, but PIE is enabled nothing sits at a fixed address.
2. At the authentication prompt, try %-prefixed tokens instead of a normal name.

3. Not every function that opens a file and prints is the real one. Check what each actually reads, and find the intake buffer's size vs. how much it reads from you.

Solution

1. At "Enter callsign," type %p instead of a name. A hex number comes back instead of plain text — your input is being read as a printf() format string, not just a name. That's the bug.
2. Use %N$p to ask for specific stack values (try %6$p through %25$p across a few fresh connections). %17$p always comes back starting with 0x55 or 0x56 — that's a real address inside the program, leaked to you.
3. Disassemble the stripped binary. The normal flow is just authenticate → receive signal. The function that actually opens and prints the flag is never called on its own — you have to jump to it yourself.
4. Three functions open a file and print something, so check which file each one actually opens. Only one opens /challenge/flag.txt — that's the real one. The other two are decoys built to look the same.
5. Work out that function's address: leaked address + 0x175 (this build's offset). Note: this offset changes on a rebuild, so it has to be re-checked against that build's own disassembly.
6. The next prompt reads 200 bytes into a buffer that only holds 64 — a classic overflow. 72 bytes (the buffer plus a saved register) is enough filler to reach the return address.
7. In the same connection as the leak, send 72 bytes of filler followed by the flag function's address at "Re-transmit the recovered signal."
8. Instead of returning normally, the program jumps to the flag function, which opens and prints the flag.

Build-specific values (this shipped binary)

- Leak index: %17$p
- Offset: leaked_value + 0x175
- Overflow padding: 72 bytes

Flag

flame{ph4nt0m_transm1ss10n_fr0m_0r10n7}